Tournament Auction Platform

MongoDB Entities, Attributes and Rationale

Stack: Node.js, TypeScript, Express, Mongoose. This document lists every collection, why it exists, and its attributes (type, required or optional, validation). Indexes and relationships are summarized per entity.

Roles modelled

Owner (exactly one, global), Tournament Organizer (per tournament), Captain (per team), User (registered account) and Public visitor (no login, read-only). Only Owner and User are global roles stored on users. Organizer and Captain are tournament-scoped and live in tournament_members.

Key assumptions

- Players register using an account, so every registration is tied to a user. Private tournaments are hidden from public lists but visible to members.

- Amounts (purse, price) are stored as integers in a virtual credit unit. No real money.

- No images or videos are stored anywhere (as per requirement), so no file or blob storage is modeled.

- Account deletion anonymizes personal data but keeps tournament history intact. Owner deletion leaves the platform ownerless until a server-side re-initialization.

- Small configurations (points rules, team rules, stage config, bid increments, statistic config, contact info) are embedded, not separate collections.

- Derived data (standings, player_stats) is stored for speed but can always be rebuilt from fixtures and innings.

- Completed matches, sales and qualification decisions are locked; edits need a reason and always write an audit log.

Contents: entity overview

#	Collection	Entity	Group
1	`users`	Global User Account	Identity
2	`system_settings`	System Settings (Singleton)	Identity
3	`sessions`	Login Session / Refresh Token	Identity
4	`account_tokens`	One-Time Account Tokens	Identity
5	`account_deletion_requests`	Account Deletion Request	Identity
6	`tournament_requests`	Tournament Creation Request	Tournament
7	`tournaments`	Tournament	Tournament
8	`tournament_members`	Tournament Membership (Role in a Tournament)	Tournament
9	`teams`	Team	Tournament
10	`registration_forms`	Player Registration Form Schema	Players
11	`player_registrations`	Player Registration (Response)	Players
12	`players`	Player (Auction Pool Entry)	Players
13	`stages`	Stage (Configurable)	Structure

#	Collection	Entity	Group
14	`groups`	Group	Structure
15	`qualification_decisions`	Qualification Decision	Structure
16	`fixtures`	Fixture / Match	Matches
17	`match_innings`	Match Innings (with Batting & Bowling Lines)	Matches
18	`csv_imports`	CSV Import Batch	Matches
19	`standings`	Standings (Points Table Row)	Results
20	`player_stats`	Player Statistics	Results
21	`auctions`	Auction	Auction
22	`auction_lots`	Auction Lot	Auction
23	`bids`	Bid	Auction
24	`auction_sales`	Sale / Player Assignment	Auction
25	`purse_transactions`	Purse Ledger	Auction
26	`notifications`	Notification	System
27	`audit_logs`	Audit Log	System

1. Global User Account

Collection: `users`

What it is	One document per registered person (normal user, tournament organizer, captain, and the single Owner). Holds identity and credentials only.
Why required	Login is global (Username + Email + Password). Tournament-specific roles must not live here, otherwise one person could not be organizer in one tournament and captain in another. The Owner is a user with role 'owner'.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`username`	String	Yes	3-30 chars, letters/digits/underscore. Original casing kept for display
`usernameLower`	String	Yes	Lower-cased copy, UNIQUE. Enforces case-insensitive uniqueness
`email`	String	Yes	Lower-cased, validated format, UNIQUE
`passwordHash`	String	Yes	argon2id (or bcrypt cost 12). select:false, never returned by APIs
`role`	Enum	Yes	'user' | 'owner'. Default 'user'. Only 'user' can be set via public signup
`status`	Enum	Yes	'active' | 'pending_deletion' | 'deleted'. Default 'active'
`tokenVersion`	Number	Yes	Default 0. Incremented on password reset/deletion to kill all JWTs
`failedLoginCount`	Number	Yes	Default 0. Brute-force protection
`lockedUntil`	Date	No	Temporary lock after repeated failures
`lastLoginAt`	Date	No	Set on successful login
`deletedAt / anonymizedAt`	Date	No	Set when account deletion completes (data anonymized, not erased)
`createdAt / updatedAt`	Date	Auto	Mongoose timestamps

Indexes / unique constraints: UNIQUE usernameLower; UNIQUE email; UNIQUE partial index on {role:1} where role='owner' (hard guard: only one owner can ever exist); {status:1}

Relations: 1 user -> N sessions, memberships, registrations, notifications

2. System Settings (Singleton)

Collection: `system_settings`

What it is	Exactly one document (_id: 'global') storing platform-wide state: owner-initialization lock, bootstrap secret hash, and global switches.
Why required	Needed to implement secure one-time owner initialization and to remember whether an owner exists, without any public owner-registration or promotion endpoint.

Attribute	Type	Req.	Details / Validation
`_id`	String	Yes	Fixed value 'global'
`ownerUserId`	ObjectId (users)	No	Null until owner is initialized or after owner deletion
`ownerInitialized`	Boolean	Yes	Default false. Flips to true atomically inside a transaction
`bootstrapSecretHash`	String	No	Hash of a one-time secret from server env. Cleared once consumed
`bootstrapConsumedAt`	Date	No	When the owner was created
`ownerVacant`	Boolean	Yes	True if owner account was deleted. Re-initialization requires server-side CLI + secret, never a public API
`tournamentRequestsEnabled`	Boolean	Yes	Default true. Owner can pause new tournament requests
`defaultTimezone`	String	Yes	IANA zone, default 'Asia/Kolkata'
`updatedAt`	Date	Auto	Timestamp

Indexes / unique constraints: Primary key _id only (singleton)

Relations: Refers to users (owner)

3. Login Session / Refresh Token

Collection: `sessions`

What it is	One document per active login (device). Stores hashed refresh token metadata.
Why required	Allows logout, logout-everywhere, token rotation, and forced session invalidation when a password is reset or an account is deleted.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`userId`	ObjectId (users)	Yes	Owner of the session
`refreshTokenHash`	String	Yes	SHA-256 of refresh token, never plain text
`family`	String	Yes	Token family id for reuse detection
`userAgent / ip`	String	No	For audit and session list
`expiresAt`	Date	Yes	TTL index auto-deletes expired sessions
`revokedAt`	Date	No	Set on logout or invalidation
`createdAt`	Date	Auto	Timestamp

Indexes / unique constraints: {userId:1}; UNIQUE refreshTokenHash; TTL on expiresAt

Relations: N sessions -> 1 user

4. One-Time Account Tokens

Collection: `account_tokens`

What it is	Short-lived tokens for password reset and deletion confirmation steps.
Why required	Password reset requires Username + Email match, then a token. Deletion needs two confirmations. Tokens must be hashed, single-use, and expire automatically.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`userId`	ObjectId (users)	Yes	Target user
`purpose`	Enum	Yes	'password_reset' | 'delete_step1' | 'delete_step2'
`tokenHash`	String	Yes	Hashed token
`attempts`	Number	Yes	Default 0, max 5, then invalidated
`expiresAt`	Date	Yes	TTL index (e.g. 15 minutes)
`usedAt`	Date	No	Single-use marker

Indexes / unique constraints: UNIQUE tokenHash; {userId:1,purpose:1}; TTL on expiresAt

Relations: N tokens -> 1 user

5. Account Deletion Request

Collection: `account_deletion_requests`

What it is	Tracks the two-step deletion workflow of any user or the owner.
Why required	Spec demands 2-time confirmation. Also records how tournaments owned by the user were handled (transfer, cancel, archive) and gives an audit trail.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`userId`	ObjectId (users)	Yes	Account being deleted
`status`	Enum	Yes	'step1_pending' | 'step2_pending' | 'completed' | 'cancelled' | 'expired'
`step1ConfirmedAt`	Date	No	First confirmation
`step2ConfirmedAt`	Date	No	Second (final) confirmation
`ownedTournamentResolution`	Array	No	[{tournamentId, action:'transfer'|'archive'|'cancel', newOrganizerId}]

Attribute	Type	Req.	Details / Validation
completedAt	Date	No	Deletion + anonymization finished
expiresAt	Date	Yes	Request auto-expires if not finished

Indexes / unique constraints: {userId:1,status:1}; partial UNIQUE on userId where status in open states

Relations: 1 user -> N requests (only one open)

6. Tournament Creation Request

Collection: `tournament_requests`

What it is	A user's request to create a tournament, waiting for Owner approval.
Why required	Tournaments must only exist after the Owner approves. Keeping requests separate from tournaments avoids half-created tournament records and keeps the approval history.

Attribute	Type	Req.	Details / Validation
_id	ObjectId	Auto	Primary key
requesterId	ObjectId (users)	Yes	Person asking to organize
proposedName	String	Yes	Proposed tournament name
description	String	No	Short pitch
status	Enum	Yes	'pending' | 'approved' | 'rejected' | 'withdrawn'. Default 'pending'
reviewedBy	ObjectId (users)	No	Owner who decided
reviewedAt	Date	No	Decision time
reviewNote	String	No	Reason shown to requester
tournamentId	ObjectId (tournaments)	No	Filled on approval
createdAt	Date	Auto	Timestamp

Indexes / unique constraints: {status:1,createdAt:-1} (owner dashboard); {requesterId:1}

Relations: 1 request -> 0..1 tournament

7. Tournament

Collection: `tournaments`

What it is	The central aggregate. Holds general details, status, visibility, team limits, points rules and statistic configuration.
Why required	Every other entity hangs off a tournament. It also drives the public pages (past / current tournaments and their stage). Small configs are embedded here instead of separate collections.

Attribute	Type	Req.	Details / Validation
_id	ObjectId	Auto	Primary key
name	String	Yes	3-120 chars
slug	String	Yes	URL-safe, UNIQUE
description	String	No	Up to 5000 chars
organizerId	ObjectId (users)	Yes	Primary organizer
requestId	ObjectId (tournament_requests)	Yes	Approval that created it
visibility	Enum	Yes	'public' | 'private'. Default 'public'
status	Enum	Yes	draft | registration_open | registration_closed | auction_scheduled | auction_in_progress | tournament_in_progress | completed | cancelled
timezone	String	Yes	IANA zone for all schedules and deadlines
registrationOpensAt / registrationClosesAt	Date	Yes	Closes must be after opens
auctionScheduledAt / expectedStartDate	Date	No	Shown publicly as expected dates
rulesText	String	No	Rules and instructions

Attribute	Type	Req.	Details / Validation
contact	Object	Yes	{name, email, phone, other} visible to participants
teamConfig	Object	Yes	{totalTeams, maxPlayersPerTeam, minPlayersPerTeam, playersPerMatch, reservePlayers}
pointsConfig	Object	Yes	{win, loss, tie, noResult} default 2/0/1/1, editable anytime
nrrMode	Enum	Yes	'auto' | 'manual'. Default 'auto'
statsConfig	Array	Yes	[{key, label, metric, enabled, scope}] e.g. most runs, most wickets, custom ones
currentStageId	ObjectId (stages)	No	Public 'current stage' display
lockedAt	Date	No	Set when completed; edits then need reason + audit
version	Number	Yes	Optimistic locking, default 0
createdAt / updatedAt	Date	Auto	Timestamps

Indexes / unique constraints: UNIQUE slug; {visibility:1,status:1,registrationOpensAt:-1}; {organizerId:1}

Relations: 1 tournament -> N teams, stages, fixtures, players, auction

8. Tournament Membership (Role in a Tournament)

Collection: `tournament_members`

What it is	Links a global user to a tournament with a tournament-scoped role: organizer, co-organizer, captain, or captain volunteer.
Why required	Required role model: Captains and Organizers exist only inside one tournament. This table gives authorization checks and supports 'select captain from approved volunteers'.

Attribute	Type	Req.	Details / Validation
_id	ObjectId	Auto	Primary key
tournamentId	ObjectId (tournaments)	Yes	Tournament scope
userId	ObjectId (users)	Yes	Member
role	Enum	Yes	'organizer' | 'co_organizer' | 'captain' | 'captain_volunteer'
teamId	ObjectId (teams)	No	Required when role = 'captain'
status	Enum	Yes	'pending' | 'approved' | 'revoked'
assignedBy	ObjectId (users)	No	Organizer who approved/assigned
createdAt / updatedAt	Date	Auto	Timestamps

Indexes / unique constraints: UNIQUE {tournamentId,userId,role}; UNIQUE partial {teamId} where role='captain' and status='approved'; {userId:1}

Relations: N memberships <-> users and tournaments (many-to-many)

9. Team

Collection: `teams`

What it is	A team inside a tournament, with captain, limits, team-specific rules and live purse counters.
Why required	Teams are the unit in auction, groups, fixtures and standings. Purse balance is kept here for fast guarded updates; every change is also written to the ledger.

Attribute	Type	Req.	Details / Validation
_id	ObjectId	Auto	Primary key
tournamentId	ObjectId (tournaments)	Yes	Parent
name	String	Yes	Editable by captain or organizer. Unique within tournament
nameLower	String	Yes	For case-insensitive uniqueness
shortCode	String	No	2-5 chars
captainUserId	ObjectId (users)	No	Current captain
status	Enum	Yes	'draft' | 'pending' | 'approved' | 'rejected'
rules	Object	No	{maxPlayers, minPlayers, playersPerMatch, reservePlayers, notes} overrides tournament defaults

Attribute	Type	Req.	Details / Validation
seed	Number	No	Used for seeded group allocation
purse	Object	Yes	{initial, balance, reserved}. balance >= 0 enforced
squadCount	Number	Yes	Default 0. Counter, kept in sync with active sales
nameHistory	Array	No	[{name, changedBy, at}] small history

Indexes / unique constraints: UNIQUE {tournamentId,nameLower}; {tournamentId,status:1}; {captainUserId:1}

Relations: 1 team -> N players, sales, standings rows

10. Player Registration Form Schema

Collection: `registration_forms`

What it is	Organizer-built Google-Form-like schema: fields, validation, and custom player categories. Versioned.
Why required	Forms vary by tournament, so fields cannot be fixed columns. Storing the schema as data lets the frontend render any form and the backend validate any response.

Attribute	Type	Req.	Details / Validation
_id	ObjectId	Auto	Primary key
tournamentId	ObjectId (tournaments)	Yes	One published form per tournament
version	Number	Yes	Increment on each publish
status	Enum	Yes	'draft' | 'published' | 'closed'
fields	Array	Yes	[{key, label, type, required, order, helpText, options[], validation{min,max,minLength,maxLength,regex}}]
fields.type	Enum	Yes	text | textarea | number | email | phone | select | multiselect | radio | checkbox | date | url
nameFieldKey	String	Yes	Which field supplies the player display name
categoryFieldKey	String	No	Which field maps a player to a category
categories	Array	No	[{key, label, basePrice, minPerTeam, maxPerTeam}] e.g. Batsman, Bowler, All-rounder
publishedAt	Date	No	Locks the version

Indexes / unique constraints: UNIQUE {tournamentId,version}; partial UNIQUE {tournamentId} where status='published'

Relations: 1 form -> N registrations

11. Player Registration (Response)

Collection: `player_registrations`

What it is	One submitted response by a user to the registration form. Stored separately from the schema.
Why required	Keeps raw answers as submitted (with the form version) so later form changes do not corrupt old answers. Organizer approves before the player enters the auction pool.

Attribute	Type	Req.	Details / Validation
_id	ObjectId	Auto	Primary key
tournamentId	ObjectId (tournaments)	Yes	Parent
formId / formVersion	ObjectId / Number	Yes	Schema used
userId	ObjectId (users)	Yes	Registrant (login needed to register)
responses	Object (Mixed)	Yes	{fieldKey: value}, validated against schema before save
displayName	String	Yes	Extracted from nameFieldKey
categoryKey	String	No	Extracted category
status	Enum	Yes	'submitted' | 'approved' | 'rejected' | 'withdrawn'
reviewedBy / reviewedAt / reviewNote	ObjectId/Date/String	No	Moderation trail
submittedAt	Date	Yes	Default now

Indexes / unique constraints: UNIQUE {tournamentId,userId}; {tournamentId,status:1}

Relations: 1 registration -> 0..1 player

12. Player (Auction Pool Entry)

Collection: `players`

What it is	Approved registration converted into an auction-eligible player with base price, state and current team.
Why required	Auction, squads and statistics need a stable, lightweight player record independent of the free-form registration responses.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId`	ObjectId (tournaments)	Yes	Parent
`registrationId`	ObjectId (player_registrations)	Yes	UNIQUE source
`userId`	ObjectId (users)	Yes	Linked account (may be anonymized later)
`name`	String	Yes	Display name copy
`categoryKey`	String	No	Category
`basePrice`	Number	Yes	>= 0, from category or organizer override
`status`	Enum	Yes	available | in_auction | sold | unsold | assigned_zero | withdrawn
`teamId`	ObjectId (teams)	No	Current team
`soldPrice`	Number	No	Current price, 0 for zero-price assignment
`unsoldRounds`	Number	Yes	Default 0

Indexes / unique constraints: UNIQUE registrationId; {tournamentId,status:1}; {teamId:1}; {tournamentId,categoryKey:1}

Relations: N players -> 1 team

13. Stage (Configurable)

Collection: `stages`

What it is	One stage of the tournament: league, groups, knockout, playoff or custom. Contains its configuration and qualification rules as data.
Why required	Core requirement: no hardcoded formats. Any number of stages with organizer-defined names and order, including WC / IPL / custom playoffs.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId`	ObjectId (tournaments)	Yes	Parent
`name`	String	Yes	Organizer-defined
`order`	Number	Yes	Position, UNIQUE per tournament
`type`	Enum	Yes	'league' | 'groups' | 'knockout' | 'playoff' | 'custom'
`isPlayoffStage`	Boolean	Yes	Separate final qualification/playoff stage flag
`status`	Enum	Yes	'draft' | 'ready' | 'in_progress' | 'completed'
`entrantTeamIds`	Array	Yes	Teams participating
`config`	Object	Yes	{roundRobin:'single'|'double', numGroups, allocationMethod:'random'|'seeded'|'manual', allocateAllTeams, fixtureMode:'auto'|'manual', playoffTemplate:'wc'|'ipl'|'custom', tieBreakers[]}
`qualification`	Array	No	Rules: [{sourceType:'group_rank'|'match_winner'|'match_loser'|'stage_rank', groupId, rank, targetStageId, targetSlotKey}]
`qualifierCountMode`	Enum	No	'fixed' | 'variable_auto' | 'variable_manual'
`lockedAt`	Date	No	Set when stage completes

Indexes / unique constraints: UNIQUE {tournamentId,order}; {tournamentId,status:1}

Relations: 1 stage -> N groups, fixtures, standings

14. Group

Collection: `groups`

What it is	A named group inside a stage and the teams allocated to it.
Why required	Groups can be many, custom-named and different sizes. Separate documents allow per-group standings, qualifier counts and manual allocation.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId / stageId`	ObjectId	Yes	Parents
`name`	String	Yes	Group A or custom
`order`	Number	Yes	Display order
`teamIds`	Array	Yes	Allocated teams
`allocationMethod`	Enum	Yes	'random' | 'seeded' | 'manual'
`qualifyCount`	Number	No	Teams moving on (can differ per group)
`allocationSeedValue`	String	No	Random seed so allocation is reproducible

Indexes / unique constraints: UNIQUE {stageId,name}; {stageId,order}; {tournamentId,teamIds:1}

Relations: 1 group -> N fixtures, standings

15. Qualification Decision

Collection: `qualification_decisions`

What it is	Concrete result of a rule: which team moved from which group/match to which target slot, system or manual.
Why required	Rules are configuration; decisions are outcomes. Keeping them separate protects downstream stages and lets the organizer override, with history.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId`	ObjectId (tournaments)	Yes	Parent
`stageId / groupId / fixtureId`	ObjectId	No	Source of qualification
`teamId`	ObjectId (teams)	Yes	Qualified team
`sourceType`	Enum	Yes	'group_rank' | 'match_winner' | 'match_loser' | 'manual'
`rank`	Number	No	Rank in the group when applicable
`targetStageId / targetSlotKey`	ObjectId / String	Yes	Where the team goes
`origin`	Enum	Yes	'system' | 'manual'
`status`	Enum	Yes	'proposed' | 'confirmed' | 'revoked'
`decidedBy / decidedAt`	ObjectId / Date	No	Who confirmed
`locked`	Boolean	Yes	True once downstream fixtures have been played

Indexes / unique constraints: UNIQUE {targetStageId,targetSlotKey} partial where status != 'revoked'; {tournamentId,stageId}

Relations: Decision -> team, source stage, target stage

16. Fixture / Match

Collection: `fixtures`

What it is	A scheduled match between two teams or two not-yet-known slots, with status and final result.
Why required	Matches are the base of results, standings and statistics. Slot references (winner of M5, rank 2 of Group A) enable dynamic progression.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId / stageId`	ObjectId	Yes	Parents

Attribute	Type	Req.	Details / Validation
groupId	ObjectId (groups)	No	If group match
matchNumber	Number	Yes	UNIQUE per tournament
roundLabel	String	No	e.g. Qualifier 1, Semifinal
slotKey	String	No	Bracket position id
teamA / teamB	Object	Yes	{teamId|null, source:{type:'team'|'group_rank'|'match_winner'|'match_loser', ref}}
scheduledAt	Date	No	In tournament timezone
venue	String	No	Optional text
status	Enum	Yes	scheduled | live | completed | no_result | abandoned | cancelled
entryMode	Enum	No	'manual' | 'csv'
result	Object	No	{winnerTeamId, loserTeamId, type:'win'|'tie'|'no_result'|'super_over', marginText, summary}
resultStatus	Enum	Yes	'none' | 'draft' | 'finalized' | 'reverted'
finalizedAt / finalizedBy	Date / ObjectId	No	Finalization trail
version	Number	Yes	Optimistic locking

Indexes / unique constraints: UNIQUE {tournamentId,matchNumber}; {stageId,groupId}; {tournamentId,status:1,scheduledAt:1}; {'teamA.teamId':1}; {'teamB.teamId':1}

Relations: 1 fixture -> N innings (usually 2), N csv imports

17. Match Innings (with Batting & Bowling Lines)

Collection: `match_innings`

What it is	Innings-level scorecard: team totals plus embedded batting and bowling stat lines for each player.
Why required	Needed to store organizer-entered match details and as the source from which standings, NRR and leaderboards are recomputed. Lines are embedded because they are always read with the innings.

Attribute	Type	Req.	Details / Validation
_id	ObjectId	Auto	Primary key
tournamentId / fixtureId	ObjectId	Yes	Parents
inningsNumber	Number	Yes	1 or 2 (3-4 for super over)
battingTeamId / bowlingTeamId	ObjectId (teams)	Yes	Must differ and belong to the fixture
totals	Object	Yes	{runs, wickets, ballsFaced, extras}
batting	Array	No	[{playerId, order, runs, balls, fours, sixes, outType, strikeRate, strikeRateIsManual}]
bowling	Array	No	[{playerId, ballsBowled, runsConceded, wickets, maidens, economy, economyIsManual}]
source	Enum	Yes	'manual' | 'csv'
importId	ObjectId (csv_imports)	No	Source import
isCurrent	Boolean	Yes	False when superseded by correction
enteredBy	ObjectId (users)	Yes	Organizer

Indexes / unique constraints: UNIQUE partial {fixtureId,inningsNumber} where isCurrent=true; {tournamentId,'batting.playerId':1}; {tournamentId,'bowling.playerId':1}

Relations: N innings -> 1 fixture

18. CSV Import Batch

Collection: `csv_imports`

What it is	One uploaded CSV for a match, with validation result, row errors, and apply status.
Why required	Gives validated imports, error reporting and safe re-import: a repeated upload is detected by file hash and the old import is superseded, not duplicated.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId / fixtureId`	ObjectId	Yes	Target match
`uploadedBy`	ObjectId (users)	Yes	Organizer
`fileName / fileHash`	String	Yes	SHA-256 of content
`rawContent`	String	No	Stored if under size limit for replay
`rowCount`	Number	Yes	Rows parsed
`status`	Enum	Yes	'uploaded' | 'validated' | 'failed' | 'applied' | 'superseded' | 'reverted'
`errors`	Array	No	[{row, column, code, message}] (capped, e.g. 200)
`parsedPreview`	Object	No	Preview shown before apply
`supersedesImportId`	ObjectId	No	Previous import replaced
`appliedAt`	Date	No	When written to innings

Indexes / unique constraints: {fixtureId,createdAt:-1}; UNIQUE partial {fixtureId,fileHash} where status='applied'

Relations: N imports -> 1 fixture

19. Standings (Points Table Row)

Collection: `standings`

What it is	One row per team per stage/group with system-calculated values and optional manual overrides.
Why required	Points table is mandatory for every stage. Storing computed rows (not computing on every page view) keeps public pages fast, and keeping override fields separate preserves the system vs manual distinction.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId / stageId`	ObjectId	Yes	Parents
`groupId`	ObjectId (groups)	No	Null for league stage
`teamId`	ObjectId (teams)	Yes	Team
`played / won / lost / tied / noResult`	Number	Yes	Default 0
`system`	Object	Yes	{points, runsFor, ballsFacedFor, runsAgainst, ballsBowledAgainst, nrr, rank}
`override`	Object	No	{points, nrr, rank, reason, setBy, setAt}
`effective`	Object	Yes	{points, nrr, rank}: override if present else system
`computedAt`	Date	Yes	Last recalculation
`sourceFixtureVersions`	Object	No	Hash of fixture versions used

Indexes / unique constraints: UNIQUE {stageId,groupId,teamId}; {stageId,groupId,'effective.rank':1}

Relations: Derived from fixtures + innings

20. Player Statistics

Collection: `player_stats`

What it is	Aggregated per-player tournament stats (runs, wickets, strike rate, etc.) with system values and overrides.
Why required	Powers most runs, most wickets and other leaderboards. Aggregates avoid scanning all innings on every request.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId`	ObjectId (tournaments)	Yes	Parent
`playerId / teamId`	ObjectId	Yes	Subject
`batting`	Object	Yes	{matches, innings, runs, balls, fours, sixes, highScore, strikeRate}
`bowling`	Object	Yes	{innings, balls, runsConceded, wickets, bestFigures, economy}
`overrides`	Object	No	{fieldPath: {value, reason, setBy, setAt}}
`computedAt`	Date	Yes	Last recalculation

Indexes / unique constraints: UNIQUE {tournamentId,playerId}; {tournamentId,'batting.runs':-1}; {tournamentId,'bowling.wickets':-1}

Relations: Derived from match_innings

21. Auction

Collection: `auctions`

What it is	Auction configuration and live state for a tournament: purse, increments, timer, squad limits, status.
Why required	Real-time bidding needs one authoritative record of settings and current state to validate each bid.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId`	ObjectId (tournaments)	Yes	UNIQUE (one auction per tournament)
`status`	Enum	Yes	'draft' | 'scheduled' | 'live' | 'paused' | 'completed' | 'cancelled'
`scheduledAt / startedAt / endedAt`	Date	No	Lifecycle times
`settings`	Object	Yes	{pursePerTeam, defaultBasePrice, bidIncrementRules:[{fromAmount,increment}], timerSeconds, timerExtensionSeconds, maxUnsoldRounds, playerOrder:'random'|'category'|'manual'}
`squadLimits`	Object	Yes	{min, max} from tournament teamConfig
`currentRound`	Number	Yes	Default 1
`currentLotId`	ObjectId (auction_lots)	No	Lot being bid on
`stateVersion`	Number	Yes	Increments on every state change
`conductedBy`	ObjectId (users)	Yes	Organizer

Indexes / unique constraints: UNIQUE {tournamentId}

Relations: 1 auction -> N lots, bids, sales

22. Auction Lot

Collection: `auction_lots`

What it is	One player being offered in a specific round, with current highest bid and outcome.
Why required	Supports unsold rounds (same player offered again), timers, and a clear sold/unsold result per attempt.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`auctionId / tournamentId`	ObjectId	Yes	Parents
`playerId`	ObjectId (players)	Yes	Player offered
`round`	Number	Yes	1, 2, 3 ... for unsold rounds
`sequence`	Number	Yes	Order inside round
`status`	Enum	Yes	'pending' | 'live' | 'sold' | 'unsold' | 'skipped'
`basePrice`	Number	Yes	Snapshot at lot creation
`currentBid`	Object	No	{amount, teamId, bidId, serverSeq}
`timerEndsAt`	Date	No	Server-controlled

Attribute	Type	Req.	Details / Validation
`saleId`	ObjectId (auction_sales)	No	Filled when sold
`openedAt / closedAt`	Date	No	Lifecycle

Indexes / unique constraints: UNIQUE {auctionId,round,playerId}; UNIQUE {auctionId,round,sequence}; {auctionId,status:1}

Relations: 1 lot -> N bids, 0..1 sale

23. Bid

Collection: `bids`

What it is	Immutable record of every bid attempt, accepted or rejected.
Why required	Required for bid history, dispute resolution, replay and for enforcing valid ordering (server sequence).

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`auctionId / lotId`	ObjectId	Yes	Parents
`teamId`	ObjectId (teams)	Yes	Bidding team
`bidderUserId`	ObjectId (users)	Yes	Captain who bid
`amount`	Number	Yes	Must be >= base and >= current + increment, <= purse balance minus reserve for minimum squad
`serverSeq`	Number	Yes	Monotonic per lot
`status`	Enum	Yes	'accepted' | 'rejected' | 'retracted'
`rejectionCode`	String	No	e.g. LOW_BID, NO_PURSE, SQUAD_FULL
`idempotencyKey`	String	Yes	Prevents double submit
`placedAt`	Date	Yes	Server time

Indexes / unique constraints: UNIQUE {lotId,serverSeq}; UNIQUE {idempotencyKey}; UNIQUE partial {lotId,amount} where status='accepted'; {auctionId,teamId}

Relations: N bids -> 1 lot, 1 team

24. Sale / Player Assignment

Collection: `auction_sales`

What it is	Record of a player joining a team: auction sale, zero-price assignment after auction, or manual adjustment, with reversal support.
Why required	Reversals and zero-price assignments must never edit history. A status flag plus linked ledger rows keeps purse and squad counts consistent.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId`	ObjectId (tournaments)	Yes	Parent
`auctionId / lotId`	ObjectId	No	Null for post-auction assignments
`playerId / teamId`	ObjectId	Yes	Subject
`price`	Number	Yes	0 for zero-price assignment
`type`	Enum	Yes	'auction' | 'zero_price_assignment' | 'manual_adjustment'
`status`	Enum	Yes	'active' | 'reversed'
`assignedBy`	ObjectId (users)	Yes	Organizer or system
`reversedBy / reversedAt / reversalReason`	ObjectId/Date/String	No	Reversal trail
`createdAt`	Date	Auto	Timestamp

Indexes / unique constraints: UNIQUE partial {tournamentId,playerId} where status='active'; {teamId,status:1}

Relations: 1 sale -> 1 player, 1 team, 1-2 ledger rows

25. Purse Ledger

Collection: `purse_transactions`

What it is	Append-only ledger of every change to a team's purse.
Why required	Guarantees purse balance can always be rebuilt and verified. Needed for reversals and audits.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`tournamentId / teamId`	ObjectId	Yes	Parents
`saleId`	ObjectId (auction_sales)	No	Related sale
`type`	Enum	Yes	'initial' | 'sale_debit' | 'reversal_credit' | 'adjustment'
`amount`	Number	Yes	Signed value
`balanceBefore / balanceAfter`	Number	Yes	balanceAfter >= 0
`seq`	Number	Yes	Per-team sequence
`createdBy`	ObjectId (users)	Yes	Actor
`createdAt`	Date	Auto	Timestamp

Indexes / unique constraints: UNIQUE {teamId,seq}; {saleId:1}

Relations: N rows -> 1 team

26. Notification

Collection: `notifications`

What it is	In-app notifications to users or the owner (approvals, role assignments, deletion, auction events).
Why required	Spec requires owner notification for tournament requests, and users need to be informed of decisions.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`recipientId`	ObjectId (users)	Yes	Target
`type`	String	Yes	e.g. tournament_request, request_approved, captain_assigned, account_deleted
`title / body`	String	Yes	Short text
`entity`	Object	No	{kind, id} deep link
`readAt`	Date	No	Null = unread
`expiresAt`	Date	No	TTL cleanup
`createdAt`	Date	Auto	Timestamp

Indexes / unique constraints: {recipientId:1,readAt:1,createdAt:-1}; TTL on expiresAt

Relations: N notifications -> 1 user

27. Audit Log

Collection: `audit_logs`

What it is	Append-only history of important actions and edits, with before/after diffs.
Why required	Spec allows editing everything; audit logs preserve accountability and protect integrity of completed matches, sales and qualification.

Attribute	Type	Req.	Details / Validation
`_id`	ObjectId	Auto	Primary key
`actorId`	ObjectId (users)	No	Null for system actions
`actorRole`	String	Yes	owner / organizer / captain / system
`action`	String	Yes	e.g. fixture.finalize, fixture.revert, sale.reverse, owner.init
`entity`	Object	Yes	{kind, id}

Attribute	Type	Req.	Details / Validation
`tournamentId`	ObjectId	No	For filtering
`changes`	Object	No	{before, after} changed fields only
`reason`	String	No	Mandatory for edits to locked/completed data
`requestId / ip`	String	No	Traceability
`at`	Date	Yes	Timestamp

Indexes / unique constraints: {tournamentId:1,at:-1}; {'entity.kind':1,'entity.id':1,at:-1}; {actorId:1,at:-1}. App role has insert-only permission

Relations: Referenced by everything via entity

Relationship overview

Entities	Cardinality
users - tournament_members - tournaments	Many-to-many (user can have roles in many tournaments)
tournaments - teams	1 to N
teams - tournament_members (captain)	1 to 0..1 active captain
tournaments - registration_forms - player_registrations	1 to N versions, 1 to N responses
player_registrations - players	1 to 0..1
tournaments - stages - groups	1 to N, 1 to N
stages / groups - fixtures	1 to N
fixtures - match_innings	1 to N (normally 2)
fixtures - csv_imports	1 to N
stages / groups / teams - standings	1 to N rows (one per team)
players - player_stats	1 to 1 per tournament
tournaments - auctions - auction_lots - bids	1 to 1, 1 to N, 1 to N
auction_lots - auction_sales - purse_transactions	1 to 0..1, 1 to 1..2
users - sessions / notifications / deletion requests	1 to N
Any entity - audit_logs	1 to N (generic reference)

Deliberately embedded (no separate collection)

- Points rules, NRR mode, statistic leaderboard config and contact details: inside tournaments.
- Team-specific rules and purse counters: inside teams (ledger is separate because it grows).
- Stage configuration, qualification rules and playoff template: inside stages.
- Bid increment rules, timer and squad limits: inside auctions.
- Batting and bowling lines: inside match_innings.
- Form fields and player categories: inside registration_forms.